

SICP: Exercise 2.15

June 22, 2025

Eva Lu Ator, another user, has also noticed the different intervals computed by different but algebraically equivalent expressions. She says that a formula to compute with intervals using Alyssa's system will produce tighter error bounds if it can be written in such a form that no variable that represents an uncertain number is repeated. Thus, she says, par2 is a "better" program for parallel resistances than par1. Is she right? Why?

At first I thought this was about floating-point inaccuracies, but I think that may have been some overeager pattern matching. Let's go through an example here to see what's happening for par1 and par2. Say we have intervals $i_1 = (3.96, 4.04)$ and $i_2 = (15.84, 16.16)$, just like in exercise 2.14 (i.e., 1% uncertainty for both intervals).

Now, let's see what happens¹ with par1 first:

$$\begin{aligned} r_1 &= \frac{i_1 \cdot i_2}{i_1 + i_2} = \frac{(3.96, 4.04) \cdot (15.84, 16.16)}{(3.96, 4.04) + (15.84, 16.16)} = \frac{(3.96 \cdot 15.84, 4.04 \cdot 16.16)}{(3.96 + 15.84, 4.04 + 16.16)} \\ &= \frac{(62.7264, 65.2864)}{(19.8, 20.2)} = (62.7264, 65.2864) \cdot \left(\frac{1}{20.2}, \frac{1}{19.8} \right) \\ &= \left(62.7264 \cdot \frac{1}{20.2}, 65.2864 \cdot \frac{1}{19.8} \right) = \left(\frac{39204}{12625}, \frac{40804}{12375} \right) \end{aligned}$$

This is roughly equal to (3.1053, 3.2973).

And now for par2:

$$\begin{aligned} r_2 &= \frac{1}{\frac{1}{i_1} + \frac{1}{i_2}} = \frac{1}{\frac{1}{(3.96, 4.04)} + \frac{1}{(15.84, 16.16)}} = \frac{1}{1 \cdot \left(\frac{1}{4.04}, \frac{1}{3.96} \right) + 1 \cdot \left(\frac{1}{16.16}, \frac{1}{15.84} \right)} \\ &= \frac{1}{\left(\frac{1}{4.04} + \frac{1}{16.16}, \frac{1}{3.96} + \frac{1}{15.84} \right)} = \frac{1}{\left(\frac{125}{404}, \frac{125}{396} \right)} = 1 \cdot \left(\frac{1}{\frac{125}{396}}, \frac{1}{\frac{125}{404}} \right) \\ &= \left(\frac{396}{125}, \frac{404}{125} \right) \end{aligned}$$

This is roughly equal to (3.168, 3.232).

Interesting, so this definitely wasn't referring to floating-point shenanigans. There is a difference here in the math itself, so these expressions really aren't equivalent at all (taking our interval arithmetic into account). The question is where the additional uncertainty (three times as much) in par1 compared to par2 comes from.

Trying out a few things in ex1-14.rkt, it seems like divisions roughly sum the respective uncertainties of their input intervals. In par1, we divide an interval with 1% uncertainty with one having 2% uncertainty, leading to a final result of 3% uncertainty. On the other hand, par2 never does any divisions using an interval with an uncertainty wider than 0%, so our final result has the original 1% uncertainty.

I can't really intuitively explain how/where exactly the two uncertainties get added together *within* div-interval, but I'm also getting a bit tired of this problem and will thus call it a day here.

¹Note that I will not be doing any rounding here, since this is a question about exactness.